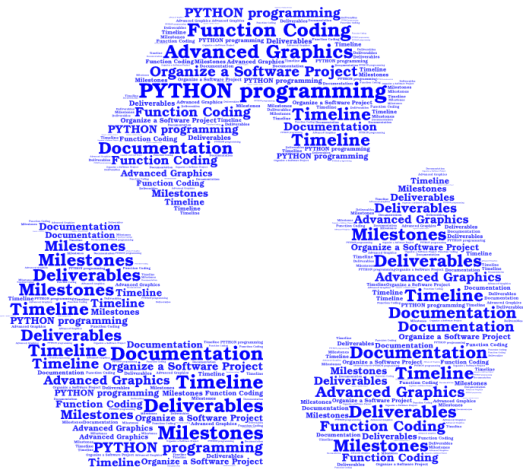




Python Programming: Work Package 3 – Battery

Lecture: Modelling an Simulation

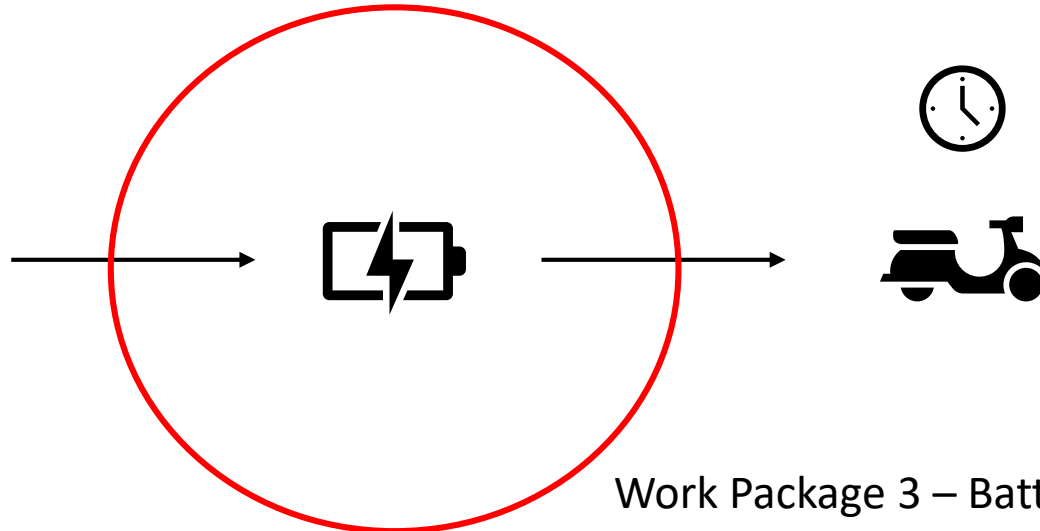
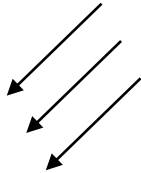
This is a compulsory elective module
of
**Technology of Circular
Economy**
(Master of Engineering)

Dr. Uwe Reimer
Senior Scientist

Overview of the project

Design a PV system which collects energy from the sun and stores it in a battery. An electric scooter shall be charged with this energy for commuting to work (daily drive cycle).

Results shall be presented for the city of Emden and second location in a different country.



Work Package 3 – Battery

Project Plan

		week						
Work package		1	2	3	4	5	6	
	Planning	*	M 1					
WP 1	Scooter							
WP 2	Solar panel							
WP 3	Battery					*	M 2	
WP 4	System Design							
WP 5	Calculate results						*	M 3
	Write report							
	List of Deliverables			List of Milestones				
	D 1	Software requirements		M 1	Plan ready			
	D 2	Scooter model		M 2	All models ready			
	D 3	PV model		M 3	Results ready			
	D 4	Battery model						
	D 5	System model						

Work package 3 / WP 3: Battery

Input:

1. Battery capacity: **M**
2. Solar generated energy per day (Charging): **S**

Output:

1. Power distribution to the scooter (Discharging): **C**

Assumptions:

1. Battery temperature: **W**
2. Battery life: **C**

Deliverable D3: State of charge (%) (Battery storage)

This was the summary from the planning phase.

Now, let's think about how we should create the model.



Actions:

- Charging
- Discharging

Information:

- Max. capacity
- How much energy is stored?
- State of battery (% charged)

Battery – what do we really need?



Actions:

- Charging
- Discharging

} functions

Information:

- Max. capacity
- How much energy is stored?
- State of battery (% charged)

} variables

(You can always add more, but this would be the core functionality.)

How to write the battery program?

Create a new file and save it as 'bat.py'.

First, we define the variables.

```
7  capamax = 480.0
8  capa = 0.0
9  state = 0.0
10
```

```
# max capacity / Wh
# how much is stored / Wh
# percent charged
```

Now, we add our first function.

If the battery is charged, the model variables need to be updated.

```
12  def charge(p):  
13      # charging the battery  
14      global capa, capamax, state  
15  
16      capa = capa + p  
17      state = 100.0 * capa / capamax  
18
```


Now, you can copy the function to create the discharge-function.

```
19 def discharge(p):  
20     # discharging the battery  
21     global capa, capamax, state  
22  
23     capa = capa - p  
24     state = 100.0 * capa / capamax
```

Everything is ready, the core functionality should be working. Let's make a first test.

Create a new file in the same directory and save it as 'test.py'.

```
8  import bat          ← import model
9
10 # read, what is maximum
11 x = bat.capamax      ← access to model variables
12 print('max. charge is ', x)
13
14 # charging battery
15 p = x
16 bat.charge(p)        ← access to model function
17
18 # test, if fully charged
19 print('battery state in %: ', bat.state)
```

Now, let's test the discharge function.

```
21 # discharge battery
22 p = 240.0
23 bat.discharge(p)
24 print('battery state in %: ', bat.state)
```

What happens, if we discharge more?

```
21 # discharge battery
22 p = 240.0
23 bat.discharge(p)
24 print('battery state in %: ', bat.state)
25
26 # try again
27 p = 400.0
28 bat.discharge(p)
29 print('battery state in %: ', bat.state)
```

What can go wrong? What needs to be considered?

-> for the function of discharging: You cannot discharge more than what is stored.

What can go wrong? What needs to be considered?

-> for the function of discharging: You cannot discharge more than what is stored.

Check, if you want to discharge to much.

- YES: Discharge all there is / battery will be empty.
- NO: You can safely discharge the requested amount.

What can go wrong? What needs to be considered?

-> for the function of discharging: You cannot discharge more than what is stored.

Check, if you want to discharge to much.

- YES: Discharge all there is / battery will be empty.
- NO: You can safely discharge the requested amount.

-> for the function of charging: You cannot exceed maximum capacity.

Check, if you want to charge to much.

- YES: Charge only the amount until you reach maximum capacity.
- NO: You can safely charge the requested amount.

```
29 def discharge(p):  
30     # discharging the battery  
31     global capa, capamax, state  
32  
33     if p > capa: # you cannot discharge more  
34         p = capa  
35         capa = 0.0  
36         state = 0.0  
37         return p  
38     else:  
39         capa = capa - p  
40         state = 100.0 * capa / capamax  
41         return p
```

→ here, we return the value that is actually discharged


```
12 def charge(p):  
13     # charging the battery  
14     # the return value gives the amount of ene  
15     global capa, capamax, state  
16  
17     # check, how much could be stored  
18     a = capamax - capa  
19  
20     if p > a: # you cannot charge more  
21         capa = capamax  
22         state = 100.0  
23         return a  
24     else:  
25         capa = capa + p  
26         state = 100.0 * capa / capamax  
27         return p
```

→ here, we return the value that is actually charged

Now, you can run the test again. -> Everything works fine.

Your job: write the software documentation for the battery model in your final report.

Imagine, you give the program to someone else, who should use it.
Which information is needed?

- 1) Purpose of this model
- 2) What is input and output?
- 3) Flow chart is not required for this simple program.
- 4) Test case/ example usage

➡ We have achieved Deliverable D4 'Battery Model'!

➡ We have successfully reached Milestone M2 'All models ready'!

